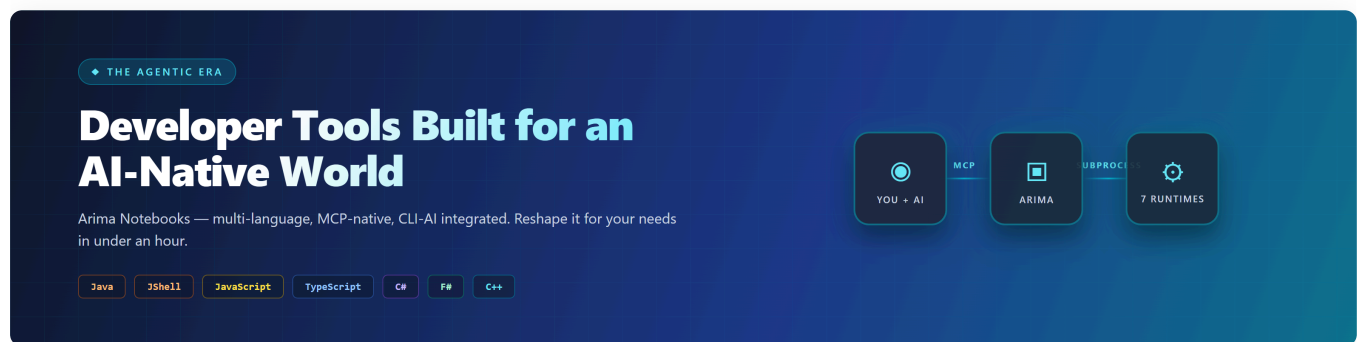


Our Developer Tools Were Built for a Pre-AI World. It's Time to Rethink Them.

Why I built Arima Notebooks — a ground-up modernization of the notebook as a local, MCP-native, multi-language execution plane shared by people and AI agents.

By Suresh Chande · ~5 min read



A Quick Acknowledgement

Jupyter is one of the great open-source achievements of the last fifteen years. I have used it, taught with it, and built on it. Nothing in this article is about replacing it.

This is about something different: the notebook — write code, run it, see results instantly — is one of software's best ideas, but it barely evolved while the world changed around it. Two shifts now matter: real work is deeply polyglot, and code is becoming a team sport with AI agents working beside humans on the same artifact.

That's the thesis. The experiment is **Arima Notebooks**, part of the Arima platform: open-source, free, local-first, and built to be reshaped.

The Two Shifts Most Teams Haven't Adjusted To

Look at any developer tool on your machine and ask:

what assumption was it built around?

- Your IDE assumes a human reading one file at a time
- Your terminal assumes a human typing one command at a time
- Your notebook assumes a human running one cell at a time
- Your package manager assumes a human deciding what to install

These are good designs — for the user they had in mind. But that user now works across many strong languages and has a partner: an AI that can read the whole file, run a sequence of commands, run every cell, and decide what to install — concurrently, on the same artifact.

If a tool wasn't built with that partner in mind, two limitations tend to show up:

1. **The AI sits outside the tool** — copy-pasting code through a side panel
2. **Customizing the tool still requires the old workflow** — fork, learn the codebase, write code by hand, file a PR, wait

The agentic era demands tools where both of those are different.

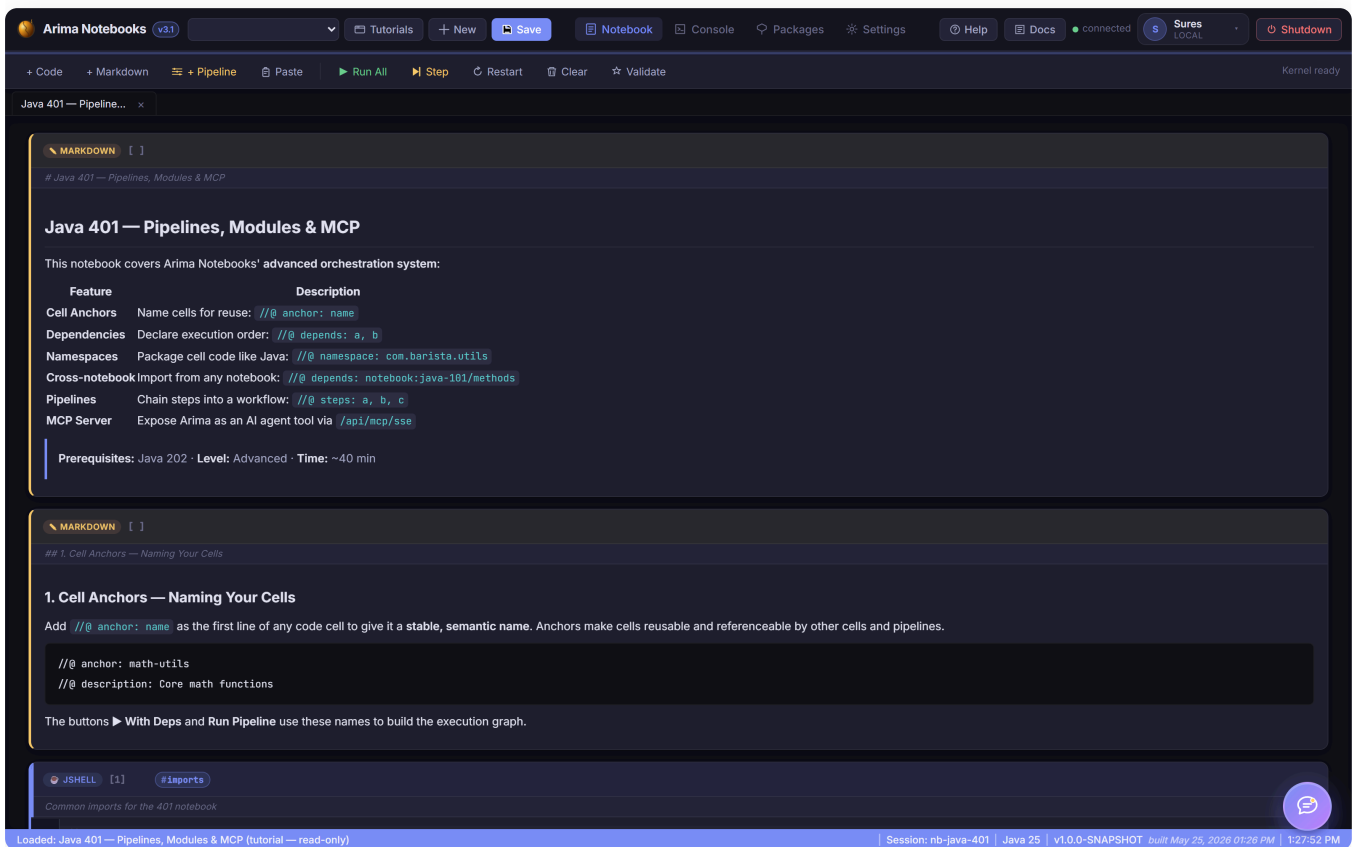
What Arima Notebooks Is

Arima Notebooks is a locally-hosted, browser-based, multi-language notebook environment — MIT-licensed and on GitHub. Think of it as a shared, cross-platform execution plane for code. It runs seven first-class language paths side-by-side in the same notebook today, with more to come:

Language	Runtime
JavaScript	Node.js subprocess
TypeScript	Node 22.6+ type-stripping + optional <code>tsc</code> type-check
C#	<code>dotnet run</code> with NuGet integration
F#	<code>dotnet fsi</code> with <code>#r "nuget:"</code> directives
C++	MSVC / GCC / Clang auto-detected
Java (full)	Per-cell <code>javac</code> compile + run
Java (JShell)	JDK's official REPL — shared state across cells

Every language gets real execution, real dependencies, and real tooling; none is a plugin or an afterthought. Built on Spring Boot, with a vanilla HTML/JS frontend (no build step), and stored as plain JSON files on disk. Nothing leaves the machine. No cloud account required.

The interesting part isn't the language list. It's three design decisions that came out of asking "what would a notebook look like if it assumed polyglot work and AI collaboration from day one?"



The Arima Notebooks UI: notebook cells with pipeline anchors, live status bar, tabs for Console, Packages, Settings, and Docs. Dark theme by default.

Decision 1 — AI in the Loop, Through Your Own CLI

Arima Notebooks runs AI as a local subprocess via whatever CLI you have authenticated:

- Claude Code CLI
- GitHub Copilot SDK (drives the local `copilot` CLI)
- Antigravity CLI (`agy`)

No second API key. No second vendor relationship. No new exfiltration path. If your security team has already cleared one of these CLIs for your developers, Arima Notebooks inherits that posture exactly.

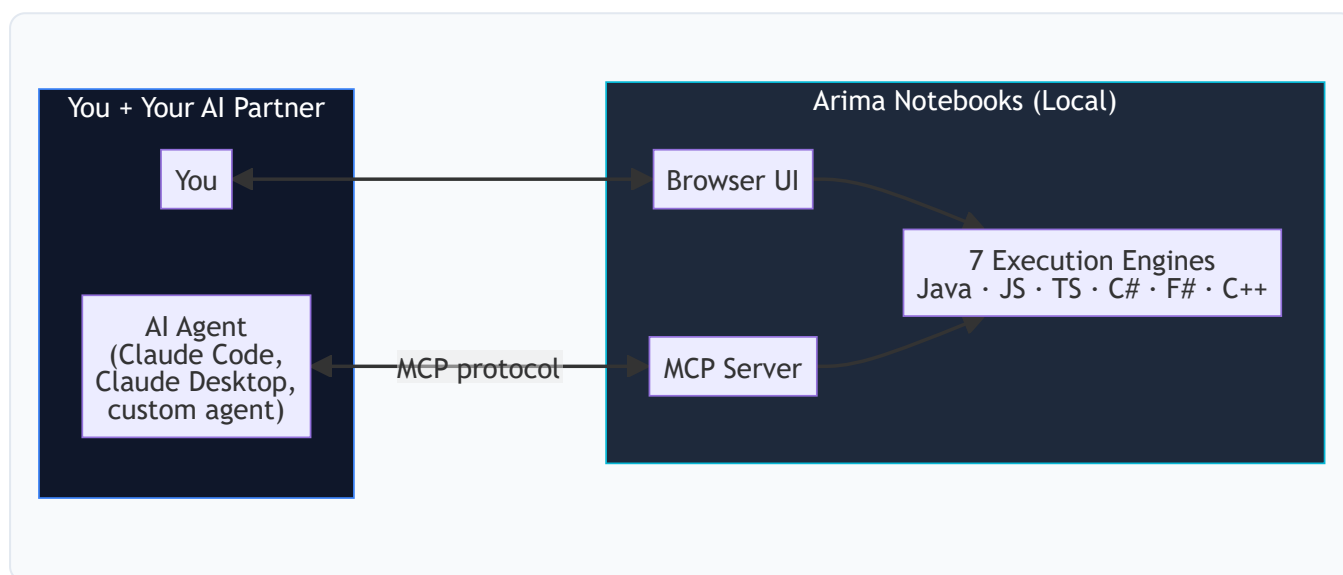
The practical effect: **the AI inside Arima Notebooks has the same powers as the AI in your terminal** — and we lean into that.

Decision 2 — The Entire System is an MCP Server

[Model Context Protocol](#) is the standard that lets AI agents drive tools in a structured way. Arima Notebooks doesn't just consume it — **it publishes itself as one.**

Every notebook, cell, execution, package install, and pipeline is exposed as an MCP tool. So you can:

- Work in the Arima Notebooks browser UI directly, **or**
- Drive Arima Notebooks from Claude Code / Claude Desktop / any MCP-aware agent, **or**
- Both, on the same notebook, simultaneously



The unlock: **an agent can prepare a notebook for you overnight.** Tell Claude Code "build an exploration notebook for our new pricing API — load sample requests, validate the schema, chart latency distributions" — and the notebook is waiting when you open Arima Notebooks in the morning.

Conversely, from inside the Arima Notebooks UI, you can attach a cell to the AI panel and ask "why is this latency spike here?" — same provider, same auth, same context.

The notebook is the shared artifact — the living document where people and agents collaborate on the same code, outputs, dependencies, and next steps. Both of you are first-class users of it.

Decision 3 — Designed to Be Reshaped in Under an Hour

This is the philosophical core, and the one I think matters most for the next decade.

Most products say:

here's what we built; submit a feature request and we'll consider it.

Arima Notebooks says:

here's what we built; if you need something else, ask your AI to add it, and it should take less than an hour.

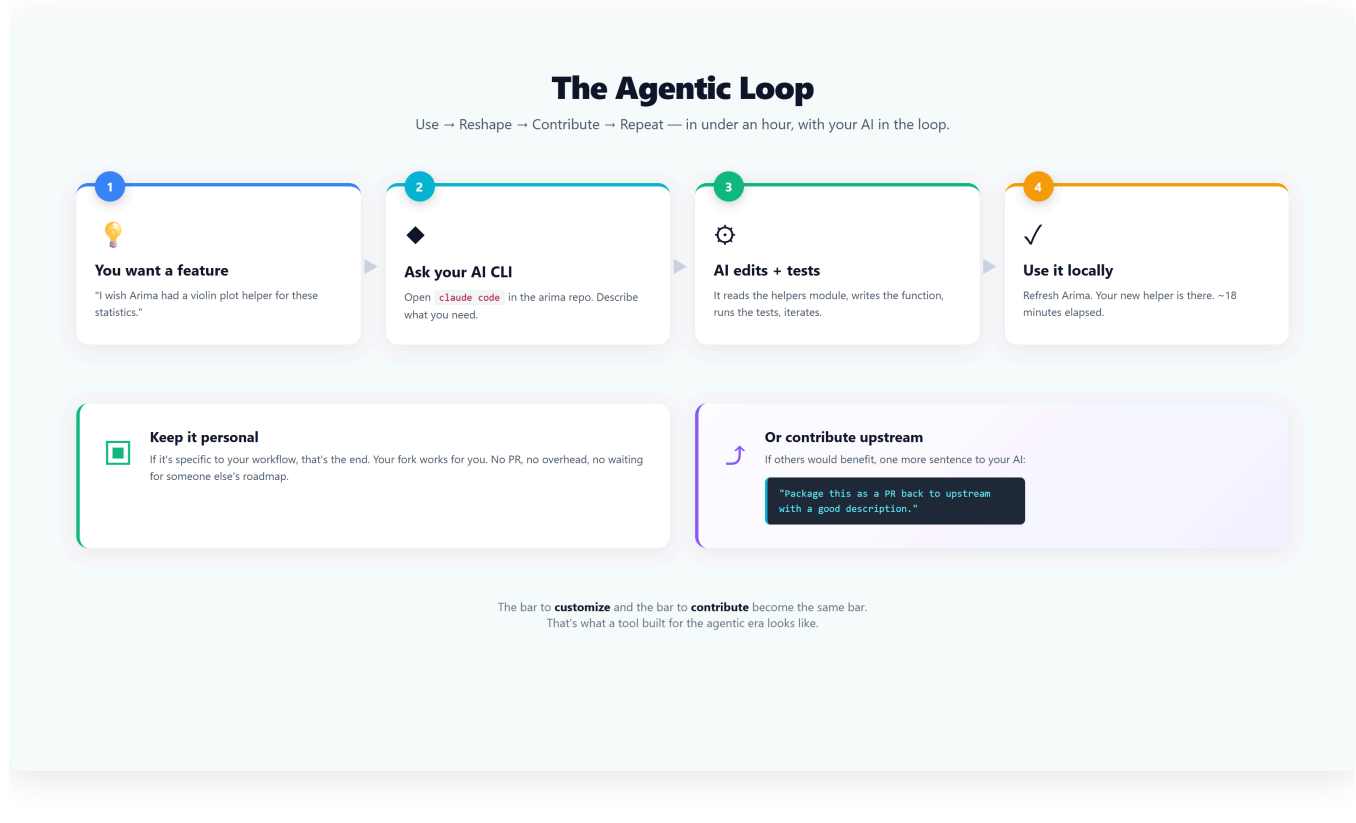
That's possible because of deliberate constraints:

- **No build step on the frontend.** Plain HTML/CSS/vanilla JS. No Webpack, no Vite.
- **Plain Java backend, no magic.** Standard Spring Boot. Any agent that can read straightforward code can extend it.
- **Subprocess-per-language.** Adding a new language = one focused execution service modeled on the existing engines.
- **Small conventions, not big frameworks.** Notebook format is JSON. Cell metadata is in `//@` annotations.

The loop becomes:

1. You want a feature → open your AI CLI in the arima repo
2. Describe what you want → AI edits the code, runs it locally
3. Works? Use it.
4. Would others benefit? *"Package this as a PR back to the upstream repo."*

The same CLI that built your local change can prepare the contribution. **The bar to give back drops to the same level as the bar to customize.**



Freedom > Lock-In

A modern developer tool shouldn't trap your work inside itself. There's no good reason a notebook authored in Arima Notebooks shouldn't open in Jupyter, or vice versa. The artifact is JSON. The cells are code. The execution model differs — but the

content

should be portable.

To be transparent: this isn't shipped yet. Arima Notebooks today reads and writes its own `.vnb` format; `.ipynb` round-tripping is planned for the next update — and it's a great first contribution for anyone who wants to try the loop described above. The principle stands: developers should be free to pick whichever tool fits the moment and move work between them without friction.

Same goes for AI providers. Same for languages. Same for package ecosystems. No language should be a plugin or an afterthought; no agent should be forced to watch from outside the artifact. **The era of "pick a tool and live inside it forever" is closing.** The era of "compose what you need, swap when convenient, shape what doesn't fit" is here.

Who Should Care

If you lead, hire, or work on a team where any of these are true:

- **Polyglot engineering teams** moving across JavaScript, TypeScript, C#, F#, C++, Java, JShell, and whatever comes next
- **Teams adopting AI agents** that need tools agents can drive, not just observe
- **Regulated industries** where SaaS notebook tools are non-starters
- **DevRel / developer education** that needs executable, shareable content
- **Open-source contributors** who want a notebook they can extend with their own languages, tools, and workflows

...Arima Notebooks is worth ten minutes of your time. And if it doesn't do what you need,
the whole pitch

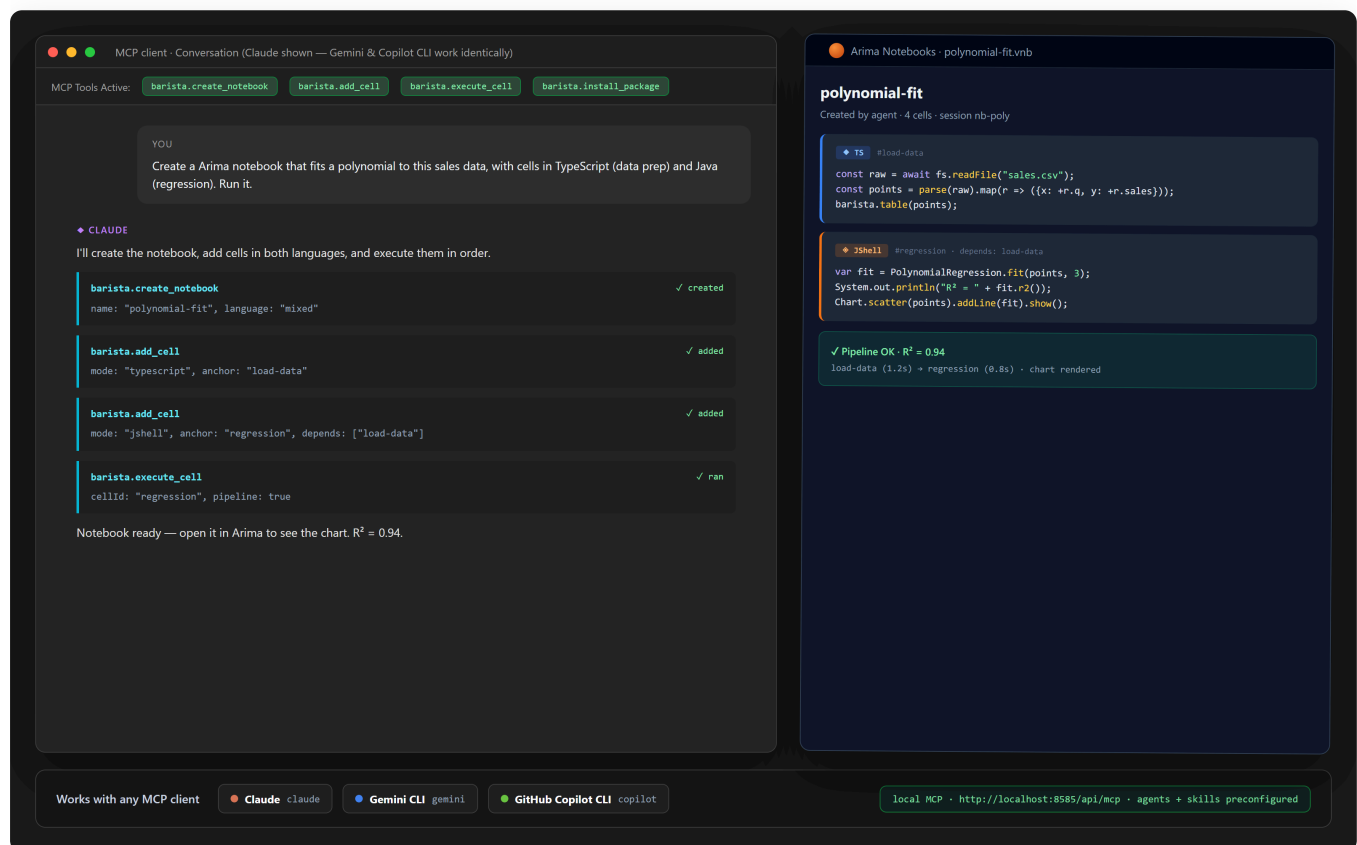
is that it should take you under an hour with an AI CLI to add what's missing.

Try It

```
git clone https://github.com/snchande/arima-notebooks.git
cd arima-notebooks
./arima      # Windows CMD — also arima.ps1 (PowerShell), arima.sh (mac/Linux)
```

Builds the JAR, starts the server at `http://localhost:8585`, opens your browser. About 30 seconds.

To drive it from an MCP-aware agent (Claude Code, Claude Desktop, custom agents): add the Arima Notebooks MCP server config — see `docs/MCP.md` in the repo.



Two surfaces, one artifact: Claude Desktop driving Arima Notebooks via MCP tools (`barista.create_notebook`, `barista.add_cell`, `barista.execute_cell`) while the resulting notebook stays live and inspectable in the Arima Notebooks UI.

What I'd Love To Hear

I built Arima Notebooks because I wanted a notebook that fit the way real work happens now: many languages, one local artifact, humans and AI agents working together. I open-sourced it because the most valuable outcome isn't Arima Notebooks staying the way I built it. It's other people forking it, asking their AI to add what they need, and (sometimes) contributing those changes back.

That loop — **use, reshape, contribute, repeat** — is what I believe open source should feel like in 2026. Tools that bend toward their users, not the other way around.

If this resonates with how you think about your team's tooling, I'd love to hear what you'd build first.

Repo: github.com/snchande/arma-notebooks **License:** MIT **Docs, MCP setup, tutorials:** all in the repo

 Repost if you think one of your developers would benefit.

#DeveloperTools #OpenSource #MCP #AgenticAI #DeveloperProductivity #SoftwareEngineering #Notebooks